

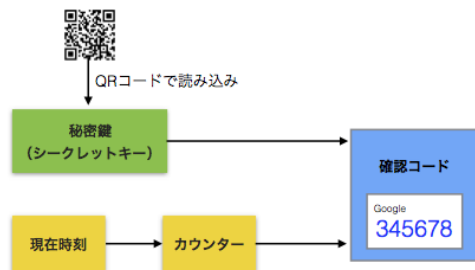
Google 認証システムの仕組み

著者：関 勝寿 公開日：2016 年 3 月 26 日 ソース：<https://sekika.github.io/2016/03/26/GoogleAuthenticator/>

Google アカウントの 2 段階認証プロセスでは、Google 認証システムをモバイル端末にインストールして、QR コード（バーコード）を読み込ませることで、30 秒ごとに変化する 6 桁の確認コードを生成することができる。通常のパスワード認証に加え、確認コードによる認証をすることで、セキュリティを高めている。Amazon, Microsoft, Facebook, Dropbox, GitHub 等多くのサービスで同じシステムが採用されている。この仕組みについて記す。

1. 概要

Google 認証システムで採用されている RFC 6238 の時間ベースのワンタイムパスワード (TOTP) は、サーバーとクライアントで共有する秘密鍵と現在時刻から、確認コードを計算するアルゴリズムである。



Google のサーバーがランダムに生成した 80 ビットの秘密鍵（シークレットキー）を QR コードあるいは 16 文字の Base32 文字列（A-Z, 2-7）としてウェブブラウザに表示し、クライアント（モバイル端末にインストールした Google 認証システムのアプリ）で読み取ることで、サーバーとクライアントに同じ秘密鍵が保存される。現在時刻は 30 秒ごとに値が変わるカウンターに変換されてから確認コードの 6 桁の数字が計算されてアプリに表示されるため、30 秒間は同じ確認コードが表示される仕組みとなっている。なお、確認コードの計算アルゴリズムについては最後に記す。

2. 秘密鍵と確認コード

ある時刻においてカウンターがサーバーとクライアントで等しければ、サーバーとクライアントで秘密鍵を共有しているため同じ確認コードが計算されるはずであるから、サーバーとクライアントで確認コードの計算結果が一致するかどうかで認証ができるとするのが TOTP アルゴリズムである。ここで、サーバーとクライアントではそれぞれ NTP のような仕組みによってある程度正確に現在時刻を得ることができるという前提であり、RFC 6238 では通信のタイムラグと時刻のずれを調整するための仕組みを実装することが推奨されている。

秘密鍵をモバイルアプリで読み込ませた端末を持っていないければ認証が成功しないことから、持っているものでアカウントを保護する仕組みであると説明される。端末を持っていないければ、漏洩した確認コードを知り得たとしても、30 秒後にはその確認コードは無効となって認証が成功しなくなるためである。しかし、たとえば中間者攻撃によって素早く漏洩した確認コードが使われれば認証に成功してしまう。30 秒間は何度でも同じ確認コードを使えるという意味ではワンタイムではない。サーバー側で最後に認証に成功した時のカウンターを記憶しておき、一致した時にはそのカウ

ンターでは 2 回目は認証に成功させないようにすれば、真の意味でワンタイムになると考えられるが、RFC 6238 ではそのような仕組みは提案されていないようである。

秘密鍵が漏洩すれば任意の時刻における確認コードを生成できるため、秘密鍵は適切なアクセス権限を設定して保存する必要があり、できれば耐タンパー性のあるデバイスに保存することが望ましいとされている。

3. TOTP に対応するサービス

Google 認証システムを使って TOTP の確認コードを生成できるサービスには、例えば次のようなサービスがある。このように多くのサービスで同じモバイルアプリを使用できることが、このシステムの利便性であろう。

- [Google](#)
- [Amazon](#)
- [Microsoft](#)
- [Twitter](#)
- [Instagram](#)
- [Facebook](#)
- [Dropbox](#)
- [Evernote](#)
- [GitHub](#)
- [WordPress](#)
- [Slack](#)
- [Discord](#)
- [Yahoo! Japan](#) - 秘密鍵の読み込み方法に注意

4. TOTP を計算するモバイルアプリ

TOTP をサポートするモバイルアプリには、例えば次のようなものがある。

- [Google 認証システム](#) (Android, iPhone, BlackBerry) - ソースコード
- [Microsoft Authenticator](#) (Android, iPhone)
- [IJ Smartkey](#) (Android, iPhone)
- [Duo Mobile](#) (Android, iPhone)
- [Token2](#) (Android, iPhone, Windows Phone)

5. 複数端末への秘密鍵の登録

サーバーが QR コードを生成した時に、複数の端末でアプリから同じ QR コードを読み込ませれば、同じ秘密鍵が登録されるため、どの端末からも同じ確認コードを生成できる。端末の紛失や故障、ソフトウェアの不具合によってモバイルアプリが起動できなくなった時のことを考えると、可能であれば 2 つ以上の端末に同じ秘密鍵を登録するのが望ましい。SMS や音声通話による 2 段階認証を設定する手段も有効である（端末を紛失しても電話番号の契約が継続していれば新しい端末で復活可能なため）。

Google 認証システムのアプリでは、アプリに登録されている秘密鍵を
QR コードによってエクスポート できる。Microsoft Authenticator では、[秘密鍵を一括してクラウドにバックアップ](#)することができる。

IJ SmartKey は、2016 年の時点でアプリに登録されている秘密鍵を QR コードでエクスポートできるほぼ唯一のアプリとして重宝して使っていたが、2020 年現在、著者が所有している iPad の中の 1 つで IJ SmartKey が起動しなくなったため、これからは Google と Microsoft のアプリを使うこととした。

6. 確認コードの計算アルゴリズム

[RFC 6238](#) の TOTP アルゴリズムは、サーバーとクライアントで共有する秘密鍵と、現在時刻から計算されるカウンターから、一意にトークン TOTP すなわち確認コードを計算するアルゴリズムであり、[RFC 4226](#) の HOTP（HMAC ベースのワンタイムパスワード）に基づいている。具体的には次のように計算する。

- ・ K を秘密鍵、TC を現在時刻（[UNIX 時間](#)）、X を時間ステップ（秒）、T0 をカウント開始時刻（UNIX 時間）、N をトークンの長さとする。また、ハッシュアルゴリズムを決める。デフォルトでは X=30, T0=0, N=6, ハッシュアルゴリズムは SHA-1 であり、Google 認証システムではこのデフォルトを使って計算をする。なお、HOTP では秘密鍵は 128 ビット以上が必要で 160 ビットを推奨としているが、Google 認証システムでは 80 ビットである。

- ・ $T = \text{floor}((TC - T0) / X)$ により、時刻 T0 からの経過時間に応じたカウンター T を 64 ビットの符号なし整数型で得る。ここで、[floor](#) は床関数であり、T を整数型としておけば通常は自動的に floor 関数が適用される。

- ・ $H = \text{HMAC-SHA-1}(K, T)$ により 20 バイトのハッシュ H を計算する。すなわち、[HMAC-SHA-1 アルゴリズム \(RFC 2104\)](#) によって秘密鍵 K とメッセージ T からハッシュ H を計算する。

- ・ 下記の Truncate 関数を使い、 $\text{TOTP} = \text{Truncate}(H)$ として 10 進数 N 桁のトークン TOTP を計算する。

ここで Truncate 関数は [RFC 4226](#) に定められている 20 バイト文字列から 10 進数 N 桁のトークンを得る次のような関数である。

- ・ 20 バイト、すなわち 160 ビット文字列 String = String[0]...String[19] から 31 ビット文字列を得る DT 関数 DT(String) を次のように定義する。String[19] の下位 4 ビットを符号なし整数に変換して Offset を得る ($0 \leq \text{Offset} \leq 15$)。次に、 $P = \text{String}[\text{Offset}] \dots \text{String}[\text{Offset}+3]$ とする。P は 32 ビットとなり、最上位ビットを除いた 31 ビットを DT(String) とする。

- ・ $\text{DT}(\text{String})$ を符号なし整数に変換した数字を Snum として、 $D = \text{Snum} \bmod 10^N$ を計算する。D は N 桁以内の正の整数となる。D が N 桁よりも少ない時には先頭に 0 を埋めて 10 進数 N 桁のトークンとしたものが、Truncate(String) である。